Setup, Run and Validation Guide

How to install the prototype, confirm it works, and validate its results on Windows

William A. Davis
CS 690 Graduate Research Project, Fall 2026
Advisor: Dr. Omar Ochoa
Updated November 10, 2026

---

1. Before You Start

You need Python 3.11 or newer. Check by opening PowerShell and running `py --version`. If it is missing, install it from python.org and tick "Add python.exe to PATH" during setup. Java is needed only for the Maven validation in Section 5.

Every command in this guide must be run from inside the project folder, the one that contains pyproject.toml, README.md and the src folder. If you downloaded the project from GitHub as a ZIP file, this is the extracted folder whose name ends in -main. The easiest way to get there is to open the folder in File Explorer, type powershell in the address bar and press Enter, which opens PowerShell already inside it. If the commands below fail with "not recognized" or "does not appear to be a Python project" errors, PowerShell is in the wrong folder.

```
cd path\to\project-folder
```

2. One-Time Installation

Create a private Python environment for the project, allow PowerShell to run its activation script for this window only, activate it, and install the project with its test tools:

```
py -m venv .venv
Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass
.venv\Scripts\Activate.ps1
pip install -e ".[dev]"
```

The prompt should now begin with (.venv). The execution-policy change lasts only until the window is closed, so it changes nothing permanently.

3. Every Time You Open a New PowerShell Window

```
cd path\to\project-folder
Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass
.venv\Scripts\Activate.ps1
```

4. Confirm the Prototype Works

First run the automated tests. The expected result is "36 passed."

```
pytest
```

Next, have the tool read all six SBOMs:

```
sbomrisk inspect data\sboms\etherpad-v1.8.6.cdx.json data\sboms\gitea-
v1.21.0.cdx.json data\sboms\juice-shop-v20.2.0.cdx.json data\sboms\pygoat-
v2.0.1.cdx.json data\sboms\webgoat-v2026.4.cdx.json data\sboms\wikijs-
v2.5.300.cdx.json
```

The output should match the following table, and every project should report zero unreachable components and zero warnings.

SBOM	Components	Links	Direct	Max depth	Cycles
etherpad-v1.8.6	953	1554	40	8	Yes
gitea-v1.21.0	231	733	110	3	Yes (normal for Go)
juice-shop-v20.2.0	689	1063	63	7	No
pygoat-v2.0.1	33	41	16	3	No
webgoat-v2026.4	191	380	37	8	No
wikijs-v2.5.300	1367	2441	154	11	Yes

Finally, export one project's dependency table and open it in Excel:

```
sbomrisk graph data\sboms\pygoat-v2.0.1.cdx.json -o pygoat.csv
start pygoat.csv
```

Each row is one package, with its depth (1 means PyGoat uses it directly), fan_in (how many packages depend on it) and fan_out (how many packages it depends on). As a spot check, Django should be at depth 1 and pycparser at depth 3, reached through cryptography → cffi → pycparser.

5. Find, Score and Rank Vulnerabilities

This is the main pipeline. It needs internet access, because it looks up each component in OSV, NVD, FIRST EPSS and the CISA KEV catalog. NVD allows only a few requests per minute without an API key, so request a free key at https://nvd.nist.gov/developers/request-an-api-key and set it in the PowerShell window before enriching. The key lasts only while that window is open and should never be saved in a file or uploaded.

```
$env:NVD_API_KEY="your-key"
```

Enrich each test project. Every answer received is saved in the cache folder, so a second run of the same command repeats the analysis without contacting the services, and adding `--offline` guarantees that it does not.

```
sbomrisk enrich data\sboms\pygoat-v2.0.1.cdx.json
sbomrisk enrich data\sboms\juice-shop-v20.2.0.cdx.json
sbomrisk enrich data\sboms\webgoat-v2026.4.cdx.json
sbomrisk enrich data\sboms\etherpad-v1.8.6.cdx.json
sbomrisk enrich data\sboms\wikijs-v2.5.300.cdx.json
sbomrisk enrich data\sboms\gitea-v1.21.0.cdx.json
```

Each run writes results\<project>-<date>\enrichment.json. Next, rank each project. Ranking works entirely from the saved results and prints the top 10 components with a one-sentence explanation of each:

```
sbomrisk rank data\sboms\pygoat-v2.0.1.cdx.json
```

The same folder then also holds ranking.csv (open it in Excel), baseline.csv (the CVSS-only ranking), not-ranked.csv and run.json. Finally, compare every project with its baseline and run the weight sensitivity analysis:

```
sbomrisk evaluate
```

This prints one line per project, with the number of new entries in the top 10, Kendall's τ, the number of the 20 weight variations that kept the ranking stable, and the share of risk held by direct dependencies, followed by whether each target in the scoring specification was met. The detailed results are written to results\evaluation-<date>. To try different weights, copy src\sbomrisk\weights.json, edit the copy, and add `--weights my-weights.json` to the rank or evaluate command.

The cache and results folders are not part of the code and should not be uploaded to GitHub.

6. Validate the Results

Quick check against the saved reference outputs

These six commands compare the parser with the saved outputs of npm, pip, Go and Maven. Each should end with RESULT: PASS. The Etherpad check matches 789 of 790 packages (99.9 percent, above the 98 percent threshold) and lists the few that differ; one of them, JSONStream, differs only in capitalization. The Maven result also lists eleven components that appear only in the SBOM, which the validation report explains.

```
python scripts\validate_graph.py npm data\sboms\juice-shop-v20.2.0.cdx.json data\
validation\juice-shop-npm-ls.json
python scripts\validate_graph.py npm data\sboms\etherpad-v1.8.6.cdx.json data\
validation\etherpad-npm-ls.json
python scripts\validate_graph.py npm data\sboms\wikijs-v2.5.300.cdx.json data\
validation\wikijs-npm-ls.json
python scripts\validate_graph.py pip data\sboms\pygoat-v2.0.1.cdx.json data\
validation\pygoat-pipdeptree.json
python scripts\validate_graph.py go data\sboms\gitea-v1.21.0.cdx.json data\
validation\gitea-go-list.json
python scripts\validate_graph.py maven data\sboms\webgoat-v2026.4.cdx.json data\
validation\webgoat-mvn-tree.txt
```

Independent Maven check

This optional check regenerates the Maven reference output on your own machine instead of using the saved copy. It needs Java 25, which can be installed with `winget install EclipseAdoptium.Temurin.25.JDK` or downloaded from adoptium.net. After installing, open a new PowerShell window and confirm that `java -version` reports 25. WebGoat includes its own Maven launcher, so Maven does not need to be installed separately.

```
$project = Get-Location
cd ..
Invoke-WebRequest https://github.com/WebGoat/WebGoat/archive/refs/tags/v2026.4.zip -
OutFile webgoat.zip
Expand-Archive webgoat.zip -DestinationPath .
cd WebGoat-2026.4
.\mvnw.cmd -B "-Denforcer.skip=true" dependency:tree "-Dscope=runtime" "-
DoutputFile=$project\mvn-tree.txt"
cd $project
```

```
python scripts\validate_graph.py maven data\sboms\webgoat-v2026.4.cdx.json mvn-
tree.txt
```

The expected result is 179 of 179 components matched at the same depth, plus eleven components listed as appearing only in the SBOM. The validation report explains why those eleven appear and how to confirm the explanation.

7. Regenerating the SBOMs (Optional)

The script scripts/generate_sboms.sh rebuilds all six SBOMs from the tagged releases with pinned tool versions. It requires a Linux or macOS shell with git, Node.js, Go, Java with Maven, uv and the libpq development headers. It is not needed to run or validate the prototype; it exists so the dataset can be reproduced for the report.